

Remote Fuzzing with AFL++

Relatore: Andrea Lanzi

Correlatore: Davide Rusconi

Elaborato Finale di: Ossama Yousef

Project repository: <https://github.com/osamayo/RemoteFuzzingAFL>

Sistemi Embedded

I sistemi embedded sono dispositivi elettronici progettati per svolgere funzioni specifiche all'interno di sistemi più grandi. Si trovano in moltissimi ambiti, come automobili, dispositivi medici e industriali, etc.

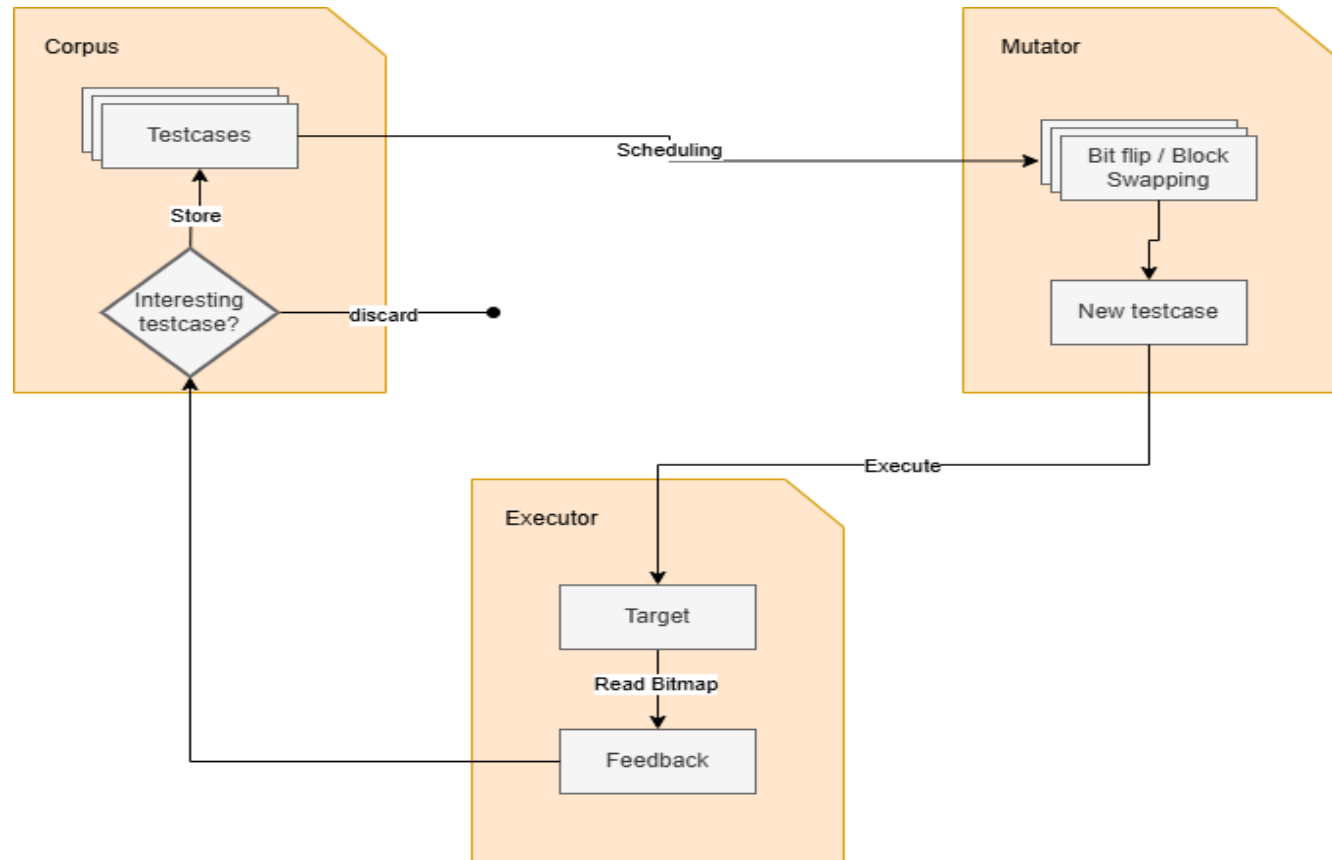
Sono sviluppati principalmente con linguaggi di basso livello, come Assembly e C/C++, per massimizzarne l'efficienza.

Vulnerabilità di basso livello:

- Stack overflow
- Heap overflow
- Use-after-free
- Arbitrary write
- Etc.

Analisi dinamici con AFL++

Il *fuzzing* è una tecnica di testing automatico che invia input generati casualmente o mutati al target, osservando eventuali crash o comportamenti anomali. È molto efficace per scoprire bug legati alla gestione della memoria e dell'input.



Fuzzing sui sistemi embedded

Nei sistemi embedded, l'assenza di un sistema operativo e le risorse limitate impedisce l'esecuzione diretta di un fuzzer. Una possibile soluzione è il remote fuzzing.

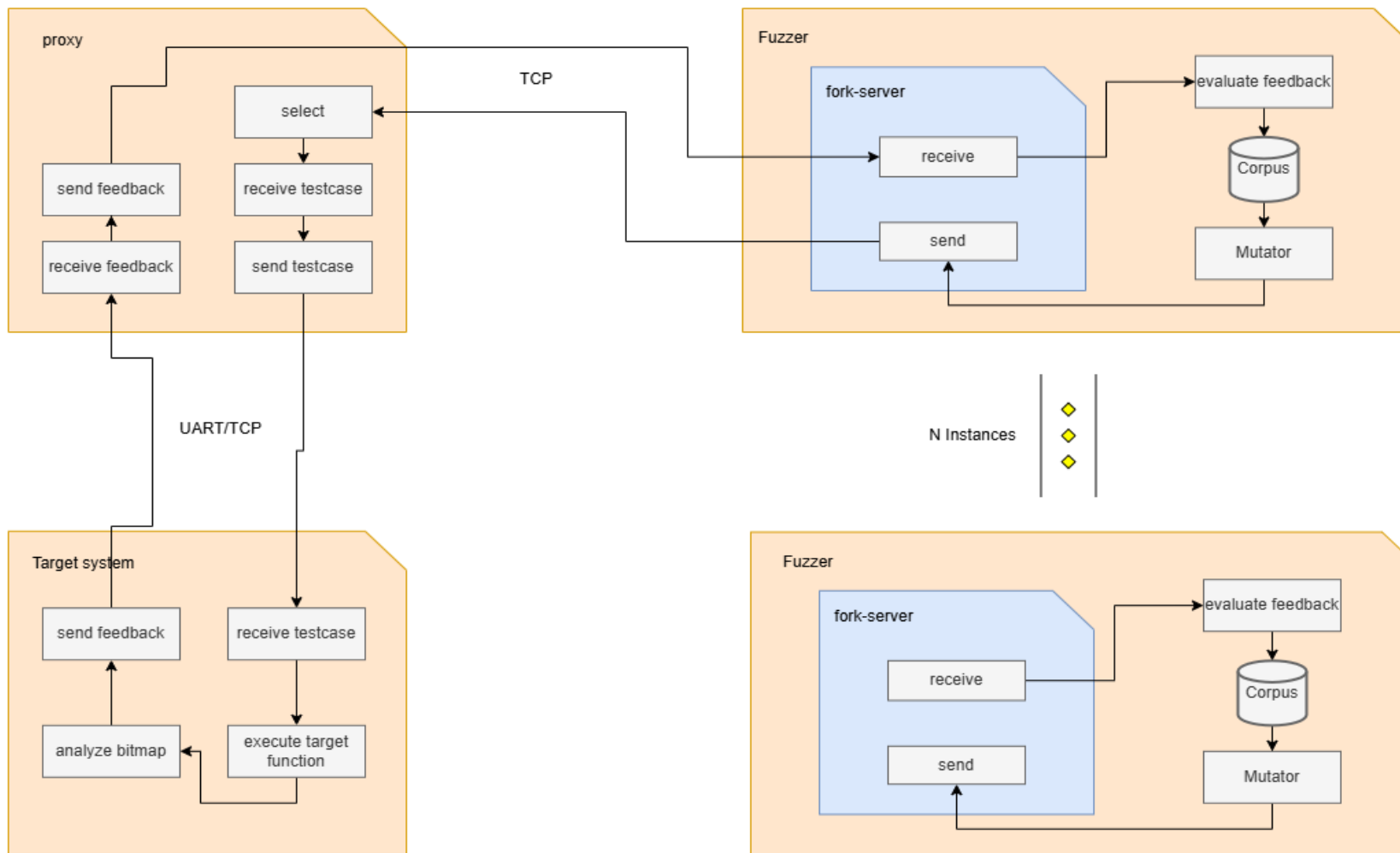
Il *Remote Fuzzing* consente di testare il sistema embedded da un host esterno, evitando la necessità di eseguire il fuzzer direttamente sul dispositivo.

Caratteristiche del framework

Il framework deve avere le seguenti caratteristiche:

- Supportare fuzzing in prallelo.
- Supportare l'in-process fuzzing.
- Analisi della bitmap direttamente sul sistema target.
- Comunicazione tramite sia TCP/IP che UART.

Architettura del framework



Risultati

Sono stati condotti tre esperimenti per valutare il framework implementato:

1. Fuzzing in locale via TCP.
2. Fuzzing remote su una board Raspberry PI via rete Internet.
3. Fuzzing su una board STM32F4 via UART.

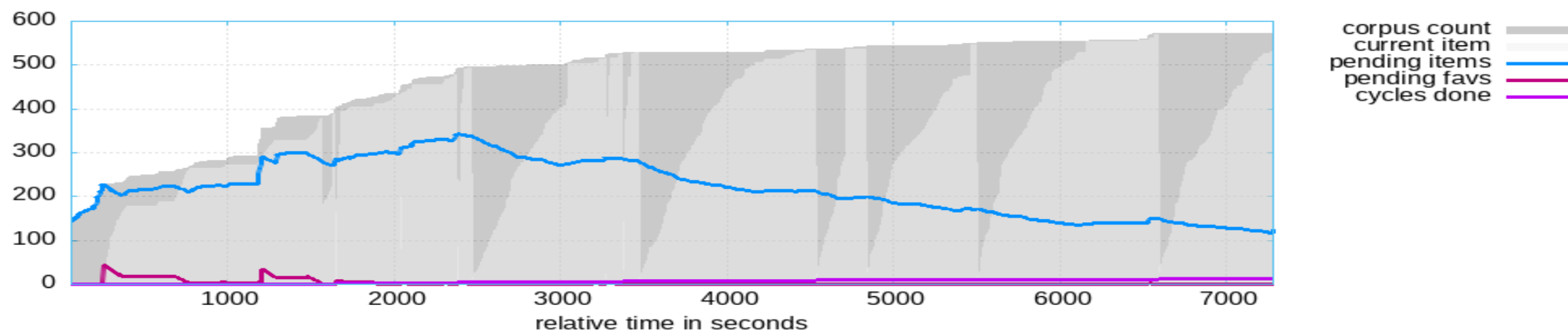
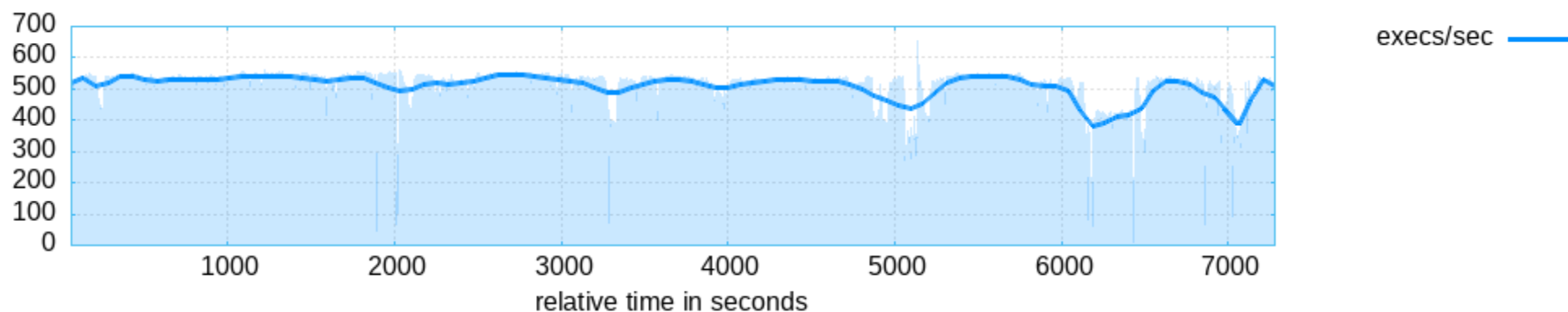
Fuzzing in locale

Instances	Remote Analysis	Exec Avg.	Corpus	Uniq Crashes	Heap
1	No	10	>590	>70	1.05MB
1	Yes	20	>750	>100	1.05MB
2	Yes	40	>1000	>100	2.10MB
4	Yes	60	>1020	>80	4.20MB
8	Yes	140	>980	>90	8.40MB
16	Yes	250	>1050	>90	16.80MB
32	Yes	450	>1770	>75	33.60MB

Fuzzing su Raspberry PI via internet

È stata condotta una campagna di fuzzing con 32 istanze di AFL++ su una board Raspberry PI tramite rete Internet.

Le istanze di AFL++ erano allocate su un host cloud a Rome mentre la scheda Raspberry PI era connessa alla rete dell'università di Milano.



Fuzzing su STM32F4

La campagna di fuzzing è stata condotta su una board STM32F4 tramite UART con una baud rate di 115200.

INSTANCES	DURATION	REMOTE ANALYSIS	EXEC AVG.	CORPUS
1	1h	No	5	>90
1	1h	Yes	12	>98

UART baudrate

Il baud rate della UART influisce direttamente sulla velocità di trasferimento dei dati, un valore più elevato consente una comunicazione più rapida.

```
Receiving feedback
took about 188.933042
Feedback header received
Sending testcase header
header sent
testcase length: 64
Sending testcase body 96 with padding 4
Total Sent: 100
testcase sent
Receiving feedback
took about 15.959788
Feedback header received
Sending testcase header
header sent
testcase length: 2000
Sending testcase body 2032 with padding 68
Total Sent: 2100
testcase sent
Receiving feedback
took about 189.831181
Feedback header received
Sending testcase header
header sent
testcase length: 454
Sending testcase body 486 with padding 14
Total Sent: 500
testcase sent
Receiving feedback
took about 48.999438
Feedback header received
Sending testcase header
header sent
```

UART Baud rate

Velocità di trasferimento in byte al secondo:

$$\text{Velocità (byte/s)} = \frac{115200 \text{ bps}}{10 \text{ bit per carattere}} = 11520 \text{ byte/s}$$

Numero massimo di pacchetti da 800 byte trasmessi al secondo:

$$\text{Pacchetti al secondo} = \frac{11520 \text{ byte/s}}{800 \text{ byte per pacchetto}} = 14,4 \text{ pacchetti/s}$$

Two thin, light orange lines intersect on the left side of the slide. One line is horizontal, and the other is diagonal, crossing it.

Conclusioni

I risultati sperimentali ottenuti hanno confermato la fattibilità dell'esecuzione del fuzzing da remoto con un impatto ridotto sulle prestazioni e con una stabilità elevata.

il framework di remote fuzzing può essere ulteriormente migliorato per supportare anche la strumentazione binary-based, risultando particolarmente utile nei casi in cui il codice sorgente non sia disponibile.